Assignment No. 07

- **TITLE –** In applications like stock market data analysis or real-time gaming leaderboards, sorting large datasets efficiently is crucial. Implement and compare various sorting algorithms Bubble Sort, Insertion Sort, Quick Sort, Shell Sort, and Radix Sort — to analyze their performance on the same input array. Track and display the time taken, number of swaps, and comparisons, and generate a report with the sorted output and performance insights.

- **THEORY –**

Sorting algorithms organize data in specific order. For gaming leaderboards, we use descending order to display highest scores first. Different algorithms have varying time complexity, space complexity, and performance characteristics.

- **ALGORITHMS USED:**
  1. **Bubble Sort**: Repeatedly compares adjacent elements and swaps if in wrong order. Each pass places the next largest element in its correct position.
  2. **Insertion Sort**: Builds sorted array one element at a time by inserting each new element into its proper position in already sorted part.
  3. **Selection Sort**: Finds minimum/maximum element from unsorted part and puts it at beginning. Divides array into sorted and unsorted parts.
  4. **Quick Sort**: Divide-and-conquer algorithm picks pivot element, partitions array around pivot, and recursively sorts sub-arrays.
  5. **Shell Sort**: Generalization of insertion sort that allows exchange of elements far apart. Uses gap sequence to sort elements.
  6. **Bucket Sort**: Distributes elements into buckets, sorts individual buckets, and concatenates results. Works well when input is uniformly distributed.
  7. **Radix Sort**: Non-comparative sorting that sorts data by processing individual digits. Uses counting sort as subroutine.
- **PERFORMANCE REPORT EXAMPLE**
  - **Sample Input:** Scores: [85, 42, 93, 61, 77, 56, 99, 34]
  - **Sorted Output (Descending Order):** [99, 93, 85, 77, 61, 56, 42, 34]

  1. Time Complexity:
  - Best: O(n log n) - Quick, Shell, Radix
  - Average: O(n log n) - Quick, Shell, Radix
  - Worst: $O(n^2)$ - Bubble, Insertion, Selection

  2. Space Complexity:
  - O(1) - Bubble, Insertion, Selection, Shell
  - O(log n) - Quick Sort (recursive)
  - O(n) - Bucket, Radix

  3. Performance Insights:
  - **Small Datasets (n < 50):** Insertion/Selection perform well
  - **Medium Datasets (50 < n < 1000):** Quick/Shell optimal

- **Large Datasets (n > 1000):** Radix/Bucket efficient
- **Memory Constraints:** Bubble/Insertion/Selection preferred
- **Stability Needs:** Insertion/Bucket stable algorithms

4. Key Metrics Tracked:

- Execution Time (microseconds)
- Number of Swaps
- Number of Comparisons

5. Practical Observations:

- Quick Sort generally fastest for random data
- Bubble Sort simplest but inefficient for large data
- Radix Sort excellent for integer scores
- Insertion Sort efficient for nearly sorted data
- Bucket Sort ideal for uniformly distributed scores

➢ ALGORITHMS –

1. Bubble Sort

Step 1: Start from first element
Step 2: Compare current element with next element
Step 3: If current < next, swap them (for descending order)
Step 4: Move to next pair, repeat until end of array
Step 5: Repeat process for n-1 passes
Step 6: Each pass places next largest element in correct position

2. Insertion Sort

Step 1: Start with second element as key
Step 2: Compare key with elements in sorted portion (left side)
Step 3: Shift elements greater than key to the right
Step 4: Insert key in correct position
Step 5: Move to next element, repeat until all elements sorted
Step 6: Builds sorted array from left to right

3. Selection Sort

Step 1: Find maximum element in unsorted portion
Step 2: Swap it with first unsorted element
Step 3: Move boundary between sorted and unsorted one element right
Step 4: Repeat finding max in remaining unsorted portion
Step 5: Continue until entire array sorted
Step 6: Divides array into sorted (left) and unsorted (right) parts

4. Quick Sort

Step 1: Choose pivot element (last element)
Step 2: Partition array around pivot
Step 3: All elements > pivot go to left partition
Step 4: All elements < pivot go to right partition

Step 5: Recursively apply to left and right partitions
Step 6: Combine results (pivot in final position)

5. Shell Sort

Step 1: Calculate gap sequence (3x+1)
Step 2: Start with largest gap
Step 3: Perform insertion sort on elements separated by gap
Step 4: Reduce gap size
Step 5: Repeat insertion sort with smaller gaps
Step 6: Final pass with gap=1 (regular insertion sort)

6. Bucket Sort

Step 1: Find maximum score value
Step 2: Create buckets array of size max+1
Step 3: Count frequency of each score in buckets
Step 4: Traverse buckets from highest to lowest
Step 5: Output each score according to its frequency
Step 6: Results automatically in descending order

7. Radix Sort

Step 1: Find maximum number to determine digits
Step 2: Start from least significant digit (LSD)
Step 3: Sort elements based on current digit using counting sort
Step 4: Move to next significant digit
Step 5: Repeat until all digits processed
Step 6: Final array sorted in ascending order (modified for descending)

- **CODE –**

```
#include<iostream>
#include<chrono>
using namespace std;
using namespace std::chrono;

int swapCount = 0;
int comparisonCount = 0;

void swap(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
    swapCount++;
}

void bubble(int arr[], int n){
    swapCount = 0;
    comparisonCount = 0;
```

```
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-1-i;j++){
            comparisonCount++;
            if(arr[j]<arr[j+1]){
                swap(arr[j],arr[j+1]);
            }
        }
    }
}

void insertion(int arr[], int n){
    swapCount = 0;
    comparisonCount = 0;
    for(int i=0;i<n;i++){
        int key = arr[i];
        int j = i-1;
        while(j>=0){
            comparisonCount++;
            if(arr[j]<key){
                arr[j+1] = arr[j];
                swapCount++;
                j--;
            } else {
                break;
            }
        }
        arr[j+1]=key;
    }
}

void selection(int arr[], int n){
    swapCount = 0;
    comparisonCount = 0;
    for(int i=0;i<n-1;i++){
        int mini = i;
        for(int j=i+1;j<n;j++){
            comparisonCount++;
            if(arr[j]>arr[mini]){
                mini = j;
            }
        }
        swap(arr[i],arr[mini]);
    }
}

int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = low-1;
    for(int j = low;j<high;j++){
        comparisonCount++;
        if(arr[j]>pivot){
            i++;
```

```
            swap(arr[i],arr[j]);
        }
    }
    swap(arr[i+1],arr[high]);
    return i+1;
}

void quick(int arr[],int low, int high){
    if(low<high){
        int pi = partition(arr,low,high);
        quick(arr,low,pi-1);
        quick(arr,pi+1,high);
    }
}

void shell(int arr[], int n){
    swapCount = 0;
    comparisonCount = 0;
    int h = 1;
    while(h<n/3){
        h = 3*h +1;
    }
    while(h>=1){
        for(int i=h;i<n;i++){
            int j=i;
            int temp = arr[i];
            while(j>=h){
                comparisonCount++;
                if(arr[j-h]<temp){
                    arr[j] = arr[j-h];
                    swapCount++;
                    j -= h;
                } else {
                    break;
                }
            }
            arr[j] = temp;
        }
        h=h/3;
    }
}

void bucket(int arr[], int n){
    swapCount = 0;
    comparisonCount = 0;
    int maxc = arr[0];
    for(int i = 1;i<n;i++){
        comparisonCount++;
        if(arr[i]>maxc){
            maxc = arr[i];
        }
    }
```

```
    int bucket[maxc+1]={0};
    for(int i=0;i<n;i++){
        bucket[arr[i]]++;
    }
    int index = 0;
    for(int i=maxc;i>=0;i--){
        while(bucket[i]>0){
            arr[index++] = i;
            bucket[i]--;
            swapCount++;
        }
    }
}

void radix(int arr[], int n){
    swapCount = 0;
    comparisonCount = 0;
    int bucket[n];
    int maxc = arr[0];
    for(int i = 1;i<n;i++){
        comparisonCount++;
        if(arr[i]>maxc){
            maxc = arr[i];
        }
    }
    int pos = 1;
    int pass = 1;
    while(maxc/pos>0){
        int count[10]={0};
        for(int i=0;i<n;i++){
            int digit = (arr[i]/pos)%10;
            count[digit]++;
        }
        for(int i=8;i>=0;i--){
            count[i] += count[i+1];
        }
        for(int i=n-1;i>=0;i--){
            int digit = (arr[i]/pos)%10;
            bucket[--count[digit]] = arr[i];
            swapCount++;
        }
        for(int i=0;i<n;i++){
            arr[i] = bucket[i];
        }
        pos*=10;
    }
}

void printArray(int arr[], int n) {
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
```

```
    cout << endl;
}

int main() {
    int n;
    cout << "Enter the number of players: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " scores: ";
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    int choice;
    do {
        cout << "\nGAMING LEADERBOARD SORTING" << endl;
        cout << "1. Bubble Sort" << endl;
        cout << "2. Insertion Sort" << endl;
        cout << "3. Selection Sort" << endl;
        cout << "4. Quick Sort" << endl;
        cout << "5. Shell Sort" << endl;
        cout << "6. Bucket Sort" << endl;
        cout << "7. Radix Sort" << endl;
        cout << "8. Exit" << endl;
        cout << "Choose sorting method: ";
        cin >> choice;

        int temp[n];
        for(int i = 0; i < n; i++) {
            temp[i] = arr[i];
        }
        cout << "\nOriginal scores: ";
        printArray(arr, n);

        auto start = high_resolution_clock::now();

        switch(choice) {
            case 1:
                bubble(temp, n);
                cout << "Leaderboard (Bubble Sort): ";
                printArray(temp, n);
                break;

            case 2:
                insertion(temp, n);
                cout << "Leaderboard (Insertion Sort): ";
                printArray(temp, n);
                break;

            case 3:
                selection(temp, n);
                cout << "Leaderboard (Selection Sort): ";
                printArray(temp, n);
```

```
                break;

            case 4:
                quick(temp, 0, n-1);
                cout << "Leaderboard (Quick Sort): ";
                printArray(temp, n);
                break;

            case 5:
                shell(temp, n);
                cout << "Leaderboard (Shell Sort): ";
                printArray(temp, n);
                break;

            case 6:
                bucket(temp, n);
                cout << "Leaderboard (Bucket Sort): ";
                printArray(temp, n);
                break;

            case 7:
                radix(temp, n);
                cout << "Leaderboard (Radix Sort): ";
                printArray(temp, n);
                break;

            case 8:
                cout << "Exiting program..." << endl;
                break;

            default:
                cout << "Invalid choice!" << endl;
        }

        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(stop - start);

        if(choice >= 1 && choice <= 7) {
            cout << "Time taken: " << duration.count() << " microseconds" << endl;
            cout << "Number of swaps: " << swapCount << endl;
            cout << "Number of comparisons: " << comparisonCount << endl;
        }

    } while(choice != 8);
    return 0;
}
```

- **OUTPUT –**

```
Enter the number of players: 5
Enter 5 scores: 45
34
51
62
37

GAMING LEADERBOARD SORTING
1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Shell Sort
6. Bucket Sort
7. Radix Sort
8. Exit
Choose sorting method: 1

Original scores: 45 34 51 62 37
Leaderboard (Bubble Sort): 62 51 45 37
34
Time taken: 1999 microseconds
Number of swaps: 6
Number of comparisons: 10

GAMING LEADERBOARD SORTING
1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Shell Sort
6. Bucket Sort
7. Radix Sort
8. Exit
Choose sorting method: 2

Original scores: 45 34 51 62 37
Leaderboard (Insertion Sort): 62 51 45
37 34
Time taken: 1999 microseconds
Number of swaps: 6
Number of comparisons: 8

GAMING LEADERBOARD SORTING
1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Shell Sort
6. Bucket Sort
7. Radix Sort
8. Exit
Choose sorting method: 3

Original scores: 45 34 51 62 37
Leaderboard (Selection Sort): 62 51 45
37 34
Time taken: 997 microseconds
Number of swaps: 4
Number of comparisons: 10

GAMING LEADERBOARD SORTING
1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Shell Sort
6. Bucket Sort
7. Radix Sort
8. Exit
Choose sorting method: 4

Original scores: 45 34 51 62 37
Leaderboard (Quick Sort): 62 51 45 37
34
Time taken: 0 microseconds
Number of swaps: 11
Number of comparisons: 17

GAMING LEADERBOARD SORTING
1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Shell Sort
6. Bucket Sort
7. Radix Sort
8. Exit
Choose sorting method: 5

Original scores: 45 34 51 62 37
Leaderboard (Shell Sort): 62 51 45 37
34
Time taken: 999 microseconds
Number of swaps: 6
Number of comparisons: 8

GAMING LEADERBOARD SORTING
1. Bubble Sort
2. Insertion Sort
```

```
3. Selection Sort
4. Quick Sort
5. Shell Sort
6. Bucket Sort
7. Radix Sort
8. Exit
Choose sorting method: 6

Original scores: 45 34 51 62 37
Leaderboard (Bucket Sort): 62 51 45 37
34
Time taken: 0 microseconds
Number of swaps: 5
Number of comparisons: 4

GAMING LEADERBOARD SORTING
1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Shell Sort
6. Bucket Sort
7. Radix Sort
8. Exit
Choose sorting method: 7

Original scores: 45 34 51 62 37
Leaderboard (Radix Sort): 62 51 45 37
34
Time taken: 1000 microseconds
Number of swaps: 10
Number of comparisons: 4

GAMING LEADERBOARD SORTING
1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Shell Sort
6. Bucket Sort
7. Radix Sort
8. Exit
Choose sorting method: 8

Original scores: 45 34 51 62 37
Exiting program...
```

- **CONCLUSION –**

  This lab assignment implementation provides comprehensive analysis of sorting algorithms performance for gaming leaderboard applications, helping select optimal algorithm based on specific requirements.